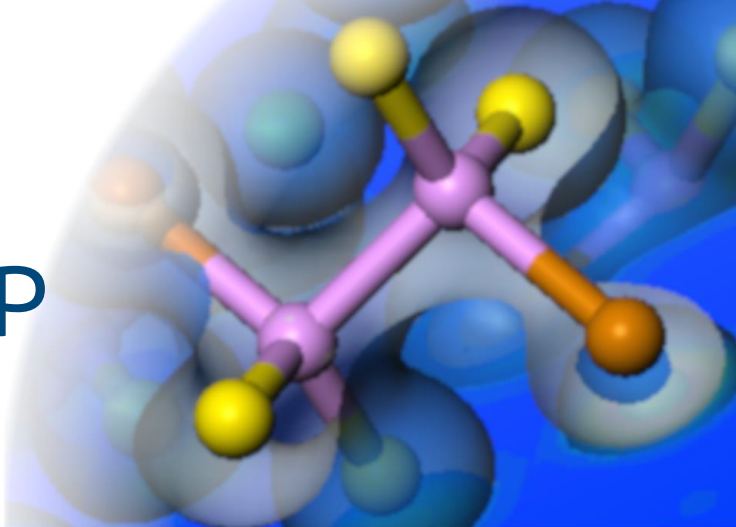




Parallel CASTEP on HPC
Ben Durham, Durham University, UK

Why run in Parallel?

CASTEP in Parallel

Introduction

Reciprocal-Space
Sampling

CASTEP in Parallel

K-Points
G-Vectors

Memory

Shared memory

Summary

We often want to simulate systems that are more complex:

- Many atoms
 - Surfaces
 - Hetero-structures
- Derived properties
 - Phonons
 - NMR
- Ensemble properties
 - Molecular Dynamics

Can't wait all year, Time : Science matters.

- CASTEP's primary parallelism uses **MPI** (Message Passing Interface)
 - Distribute work and data over N MPI processes
 - Each MPI process executes CASTEP using its data
 - Processes *communicate* as necessary
- Communication is **much** slower than computation

Recap: what does CASTEP do?

CASTEP solves the Kohn-Sham equations for electrons in a periodic array of nuclei:

$$\hat{H}_k[\rho]\psi_{bk} = E_{bk}\psi_{bk}$$

where particle b has the b th solution (“band”) at the Brillouin zone sampling point $\mathbf{k}$, and

$$\begin{aligned}\hat{H}_k[\rho] &= -\frac{\hbar^2}{2m}\nabla^2 + \hat{V}_{HXC}[\rho] + \hat{V}_{ext} \\ &= -\frac{\hbar^2}{2m}\nabla^2 + V_{HXC}[\rho](\mathbf{r}) + (V_{locps}(\mathbf{r}) + \hat{V}_{nl}) \\ &= -\frac{\hbar^2}{2m}\nabla^2 + (V_{HXC}[\rho](\mathbf{r}) + V_{locps}(\mathbf{r})) + \hat{V}_{nl} \\ &= -\frac{\hbar^2}{2m}\nabla^2 + V_{loc}[\rho](\mathbf{r}) + \hat{V}_{nl}.\end{aligned}$$

Recall that Bloch's theorem let us write:

$$\psi_{bk}(\mathbf{r}) = e^{i\mathbf{k}\cdot\mathbf{r}} u_{bk}(\mathbf{r}),$$

where $u_k(\mathbf{r})$ is periodic and $e^{i\mathbf{k}\cdot\mathbf{r}}$ is an arbitrary phase factor. We express $u_k(\mathbf{r})$ as a Fourier series:

$$\begin{aligned} u_{bk}(\mathbf{r}) &= \sum_{\mathbf{G}} c_{Gbk} e^{i\mathbf{G}\cdot\mathbf{r}} \\ \Rightarrow \psi_{bk}(\mathbf{r}) &= e^{i\mathbf{k}\cdot\mathbf{r}} \sum_{\mathbf{G}} c_{Gbk} e^{i\mathbf{G}\cdot\mathbf{r}} \\ &= \sum_{\mathbf{G}} c_{Gbk} e^{i(\mathbf{G}+\mathbf{k})\cdot\mathbf{r}} \end{aligned}$$

where c_{Gbk} are complex Fourier coefficients, and the sum is over all the **reciprocal lattice vectors**, or **G-vectors**.

The wavefunction is one of the main data objects in CASTEP:

$$\psi_{bk}(\mathbf{r}) = \sum_{\mathbf{G}} c_{\mathbf{G}bk} e^{i(\mathbf{G}+\mathbf{k})\cdot\mathbf{r}}$$

The complex coefficients $c_{\mathbf{G}bk}$ are what CASTEP is trying to compute, and take up a lot of the computer's memory.

- $\mathbf{G}$: reciprocal lattice vector ('G-vector' or 'plane-wave')
- b : band index
- $\mathbf{k}$: Brillouin zone sampling point ('k-point')

For a given density ρ , the equations for different **k**-points are independent of each other:

$$\hat{H}_k[\rho]\psi_{bk} = E_{bk}\psi_{bk}$$

where

$$\rho(\mathbf{r}) = \sum_{bk} |\psi_{bk}(\mathbf{r})|^2$$

Where does CASTEP spend its time? CASTEP

CASTEP in Parallel

Introduction

Reciprocal-Space
Sampling

CASTEP in Parallel

K-Points
G-Vectors

Memory

Shared memory

Summary

- Applying $\hat{H}_k$ to ψ_{bk}

$$\hat{H}_k[\rho] = -\frac{\hbar^2}{2m}\nabla^2 + V_{\text{loc}}[\rho](\mathbf{r}) + \hat{V}_{\text{nl}}.$$

- The kinetic energy is applied in reciprocal-space
- The local potential is applied in real-space

We need to Fourier transform between the two spaces.

- Orthogonalisation of ψ_{bk}

We need to ensure our trial bands are orthogonal to each other. We compute the overlap matrix between all pairs of bands, and invert it.

- Applying the local potential requires $\tilde{\psi}_{bk}(\mathbf{r})$
- Usually deal with $\psi_{bk}(\mathbf{G})$
- Time to transform $\psi_{bk}(\mathbf{G}) \longleftrightarrow \tilde{\psi}_{bk}(\mathbf{r})$ scales as $\sim N_G \log(N_G)$

- Applying the local potential requires $\tilde{\psi}_{bk}(\mathbf{r})$
- Usually deal with $\psi_{bk}(\mathbf{G})$
- Time to transform $\psi_{bk}(\mathbf{G}) \longleftrightarrow \tilde{\psi}_{bk}(\mathbf{r})$ scales as $\sim N_G \log(N_G)$
- Every band at every k-point has to be transformed
- Total time is $\sim N_G N_b N_k \log(N_G)$

- We construct the band-overlap matrix S for each $\mathbf{k}$ -point,

$$S_{bb'k} = \sum_{\mathbf{G}} \psi_{bk}^*(\mathbf{G}) \psi_{b'k}(\mathbf{G}).$$

Total time scales as $N_G N_b^2 N_k$.

- Invert S_k to find an orthogonalising transformation at each $\mathbf{k}$ -point. Total time scales as $N_b^3 N_k$.
- Apply transformation to get orthogonal bands. Total time scales as $N_G N_b^2 N_k$.

As we simulate larger and larger systems, N_G and N_b increase and N_k decreases (for very large cells $N_k = 1$).

- Time for Fourier transforms scales as $N_G \log N_G N_b N_k$.
For very large systems $\sim N_G N_b$.
- Time for orthogonalisation scales as $N_G N_b^2 N_k$.
For very large systems $\sim N_G N_b^2$.

→ Orthogonalisation dominates in large calculations.

CASTEP in Parallel

Introduction

Reciprocal-Space
Sampling

CASTEP in Parallel

K-Points
G-Vectors

Memory

Shared memory

Summary

As the simulation system gets bigger and bigger, the orthogonalisation time dominates.

We want to be able to use more computer cores in our calculation to speed it up – how can we do this?

- Equations at different **k**-points are almost entirely independent of each other
 - give each core a subset of the **k**-points.
 - each core solves a subset of Kohn-Sham equations
- Cores only communicate when constructing the density

$$\rho(\mathbf{r}) = \sum_{bk} |\psi_{bk}(\mathbf{r})|^2$$

Default	
Parallelism	Priority
k-points	1
G-vectors	2
bands	DEVEL_CODE only

- Computational workload is equivalent for each k-point
 - **ideal load balancing**

- Computational workload is equivalent for each k-point
 - **ideal load balancing**
- Knowledge of all k-points needed only when computing density and Fermi energy
 - **infrequent communications**

$$\rho(\mathbf{r}) = \sum_{bk} |\psi_{bk}(\mathbf{r})|^2$$

In 'cell' file

```
KPOINTS_MP_GRID 3 4 5
```

In 'cell' file

```
KPOINTS_MP_GRID 3 4 5
```

```
KPOINTS_MP_SPACING 0.1 1/ang
```

In 'cell' file

```
KPOINTS_MP_GRID 3 4 5
```

```
KPOINTS_MP_SPACING 0.1 1/ang
```

```
BLOCK KPOINTS_LIST  
0.0 0.0 0.0 1.0  
0.25 0.25 0.25 0.25  
0.5 0.5 0.5 0.75  
ENDBLOCK KPOINTS_LIST
```


- Generally, CASTEP will *not* explicitly compute the total number of k-points implied by the k-point grid or k-point density in your cell file

- Generally, CASTEP will *not* explicitly compute the total number of k-points implied by the k-point grid or k-point density in your cell file
- Use `dryrun` to check input files for parameters and syntax errors

```
% castep --dryrun <seed>  
% grep "kpoints" <seed>.castep
```

N.B. Search for "kpoints" NOT "k-points"

```
-----  
k-Points For BZ Sampling  
-----  
Number of kpoints used = 8
```

- Generally, CASTEP will *not* explicitly compute the total number of k-points implied by the k-point grid or k-point density in your cell file
- Use `dryrun` to check input files for parameters and syntax errors

```
% castep --dryrun <seed>  
% grep "kpoints" <seed>.castep
```

N.B. Search for "kpoints" NOT "k-points"

```
-----  
k-Points For BZ Sampling  
-----  
Number of kpoints used = 8
```

- Best practice: number of MPI processes = factor/multiple of number of k-points

The TiN simulation is a standard CASTEP benchmark

- 33 atoms
- 8 **k**-points
- 164 bands
- 10,972 **G**-vectors

How does k-point parallelism perform?

k-point parallelism in action

CASTEP in Parallel

Introduction

Reciprocal-Space
Sampling

CASTEP in Parallel

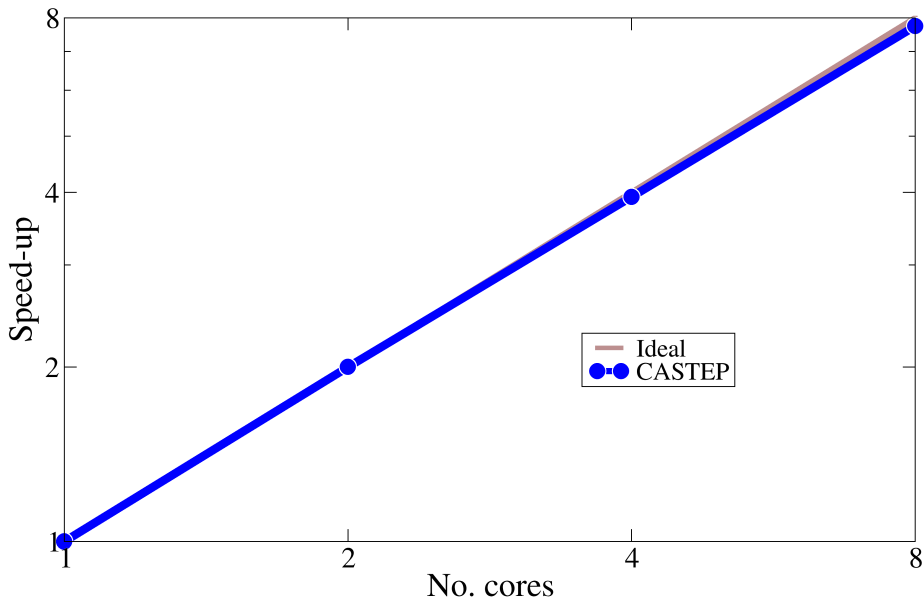
K-Points

G-Vectors

Memory

Shared memory

Summary



- **k**-point parallelism is almost perfect.
- As simulations get bigger, N_k gets smaller.
→ the bigger the simulation, the fewer the cores we can use!

- Most fine-grained form of parallelism: $N_G \sim 10k$ to $\sim 100k$
- Much of CASTEP computation scales with G-vectors $\rightarrow$ less work *and* less memory
- As simulation size increases, N_G also increases
- **Frequent communications limit efficiency** – *inter*-node comms much slower than *intra*-node comms

- In **'param' file:**

```
cut_off_energy: 500 eV
```


- In 'param' file:

```
cut_off_energy: 500 eV
```

- Defined w.r.t. kinetic energy:

$$E_{\text{cut}} = \frac{1}{2} \left| k + G_{\text{max}} \right|^2$$

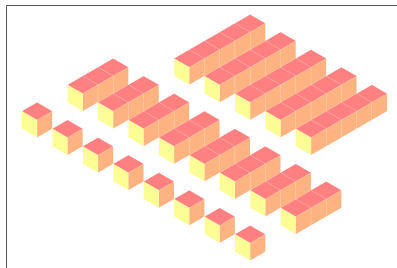
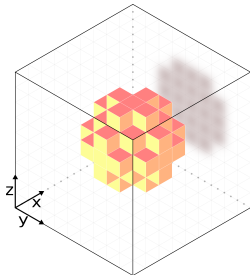
- In 'param' file:

```
cut_off_energy: 500 eV
```

- Defined w.r.t. kinetic energy:

$$E_{\text{cut}} = \frac{1}{2} \left| k + G_{\text{max}} \right|^2$$

- Form 'G-vector sphere'



G-vector parallelism in action

CASTEP in Parallel

Introduction

Reciprocal-Space
Sampling

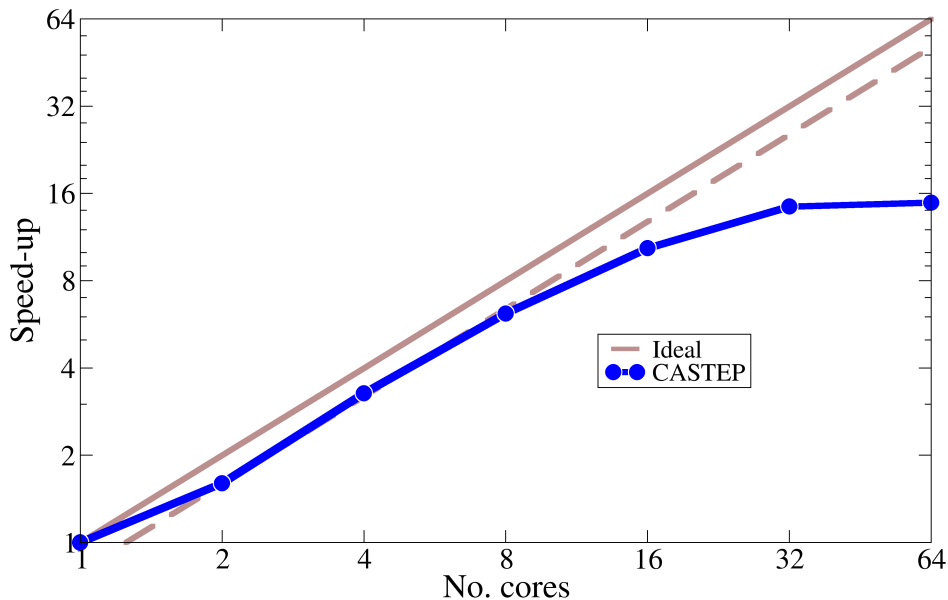
CASTEP in Parallel

K-Points
G-Vectors

Memory

Shared memory

Summary



- Reciprocal-space: each core has some of **G**-vectors
E.g. 2 **G**-vectors and 2 **r**-vectors on 2 cores

Core	G -vector	wavefunction part
A	G ₁	$\psi(\mathbf{G}_1) = c_1 e^{i\mathbf{G}_1 \cdot \mathbf{r}_1} + c_1 e^{i\mathbf{G}_1 \cdot \mathbf{r}_2}$
B	G ₂	$\psi(\mathbf{G}_2) = c_2 e^{i\mathbf{G}_2 \cdot \mathbf{r}_1} + c_2 e^{i\mathbf{G}_2 \cdot \mathbf{r}_2}$

- Real-space: each core has some **r**-vectors
E.g. 2 **G**-vectors and 2 **r**-vectors on 2 cores

Core	r -vector	wavefunction part
A	r ₁	$\tilde{\psi}(\mathbf{r}_1) = c_1 e^{i\mathbf{G}_1 \cdot \mathbf{r}_1} + c_2 e^{i\mathbf{G}_2 \cdot \mathbf{r}_1}$
B	r ₂	$\tilde{\psi}(\mathbf{r}_2) = c_1 e^{i\mathbf{G}_1 \cdot \mathbf{r}_2} + c_2 e^{i\mathbf{G}_2 \cdot \mathbf{r}_2}$

- Fourier transform: *all* **G**-vectors contribute at *all* points in real-space.

Every core has a part of the work

→ compute time scales as $\frac{1}{N_{core}}$.

In the FFT, every core communicates with every other core

→ communication time in FFT scales as N_{core}^2 .

→ as N_{core} increases, Fourier transform will dominate.

→ Eventually communication time $\approx$ compute time

→ No point using more cores (it might even be slower).

Not strictly true as of version 24 onwards.

G-vector parallelism in action

CASTEP in Parallel

Introduction

Reciprocal-Space
Sampling

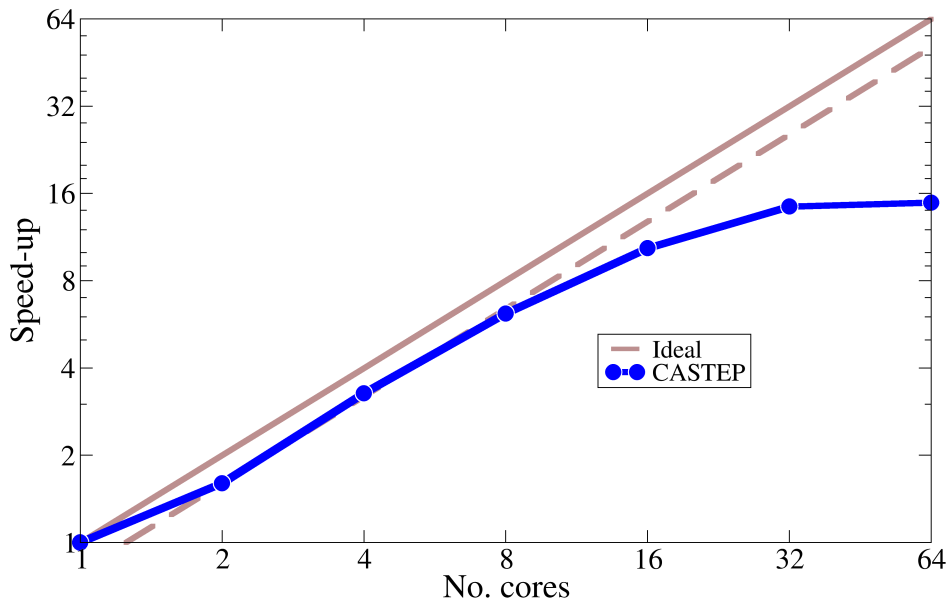
CASTEP in Parallel

K-Points
G-Vectors

Memory

Shared memory

Summary



How do you know whether you've used "too many" cores?

- CASTEP times the communication calls
- CASTEP times the whole calculation
- Reports the percentage of time that *wasn't* communication
- Also tries to give hints of things you might have missed
- Look at the end of the `.castep` file:

Overall parallel efficiency rating: Satisfactory (66%)

Data was distributed by:-

G-vector (72-way); efficiency rating: Satisfactory (69%)

k-point (8-way); efficiency rating: Excellent (93%)

Ideally, better than 70%, less than 50% is time to rethink.

- **k**-point and **G**-vector parallelisms are independent
- Can combine both to improve scaling
- E.g. if $N_k = 2$, $N_G = 9,000$ and $N_{core} = 6$:

Data	k-point 1	k-point 2
G -vectors 1-3,000	core 1	core 4
G -vectors 3,001-6,001	core 2	core 5
G -vectors 6,001-9,000	core 3	core 6

For any **k**-point, the **G**-vector data is split across 3 cores – this is **3-way G-vector parallelism**.

For any subset of **G**-vectors the k-point data is split between 2 cores – this is **2-way k-point parallelism**

k+G parallelism in action

CASTEP in Parallel

Introduction

Reciprocal-Space
Sampling

CASTEP in Parallel

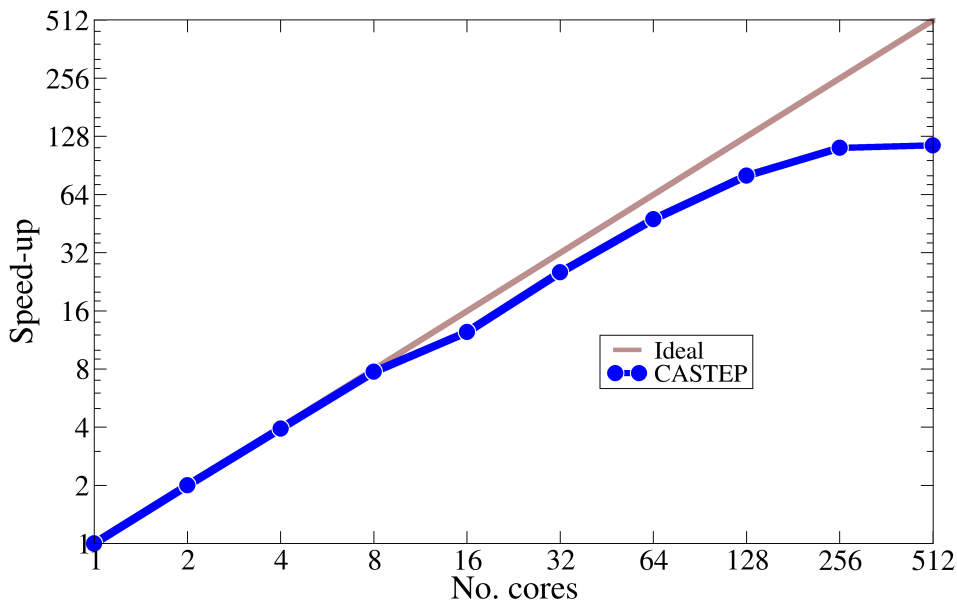
K-Points

G-Vectors

Memory

Shared memory

Summary



The Al₂O₃ surface simulation is a larger standard benchmark

<http://www.castep.org/CASTEP/Al3x3>

- 270 atoms
- 2 **k**-points
- 778 bands
- 88,184 **G**-vectors

k+G parallelism in action

CASTEP in Parallel

Introduction

Reciprocal-Space
Sampling

CASTEP in Parallel

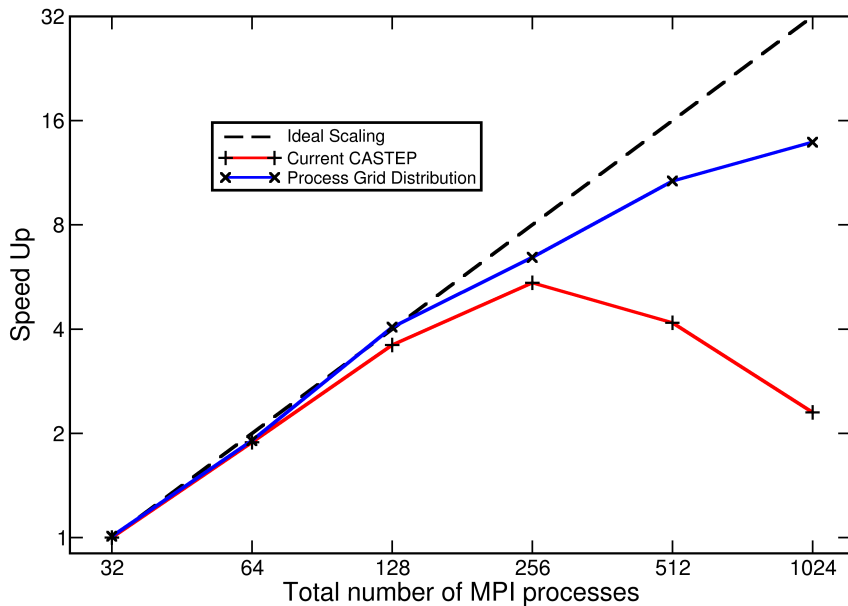
K-Points

G-Vectors

Memory

Shared memory

Summary



CASTEP always defaults to using as much **k**-point parallelism as it can, and then uses **G**-vector parallelism across any other cores.

E.g. if $N_k = 35$

Cores	Parallel distribution
70	Each pair of cores gets one k -point G -vectors are distributed within each pair
36	One core left idle; CASTEP uses 35 cores
35	Each core gets one k -point
21	Cores split into 7 triplets Each triplet of cores gets 5 k -points G -vectors are distributed within each triplet
5	Each core gets 7 k -points

You can help CASTEP:

- If $N_{\text{core}} < N_k$, try to have N_{core} a factor of N_k
→ Each core will get the same number of **k**-points.
- If $N_{\text{core}} > N_k$, try to have N_{core} a multiple of N_k
→ Each group of cores will get one **k**-point → CASTEP will use **G**-vector parallelism within each group.

If you can't do this, try to have a high common factor between N_{core} and N_k .

You can check how many **k**-points CASTEP wants to use, with a “dryrun”, e.g. for a calculation called “Si8”

```
castep --dryrun Si8
```

Time is not the only computer resource we care about. A computer's memory (RAM) is also limited.

During any CASTEP calculation:

- Lots of intermediate data is generated
- Some of this data is not needed very often; should we:
 - Store it to reuse? (Fast, needs RAM)
 - Store it on hard disk to reuse? (Slower, needs disk)
 - Delete it and recompute if needed (Slow, no RAM)

Time is not the only computer resource we care about. A computer's memory (RAM) is also limited.

During any CASTEP calculation:

- Lots of intermediate data is generated
- Some of this data is not needed very often; should we:
 - Store it to reuse? (Fast, needs RAM)
 - Store it on hard disk to reuse? (Slower, needs disk)
 - Delete it and recompute if needed (Slow, no RAM)
- Controlled in the **.param** file:
 - `opt_strategy_bias : +3 ! fastest`
 - `opt_strategy_bias : 0 ! compromise (default)`
 - `opt_strategy_bias : -3 ! lowest memory`
- Disk read/writes are **very slow on network drives**

Some calculations proceed one **k**-point at a time, e.g.:

- Spectral calculations, e.g.
 - Bandstructures
 - DOS
 - EELS
- NMR

After each **k**-point is done, CASTEP can write output. **k**-point parallel is fast, but needs data at all **k**-points

G-vector parallel may be slower, but only needs data at one **k**-point at a time.

You can force **G**-vector parallelism in the `.param` file:

```
data_distribution : gvector
```


How do you know how much memory CASTEP needs?

- CASTEP estimates the calculation's memory requirements per MPI process:

----- MEMORY AND SCRATCH DISK ESTIMATES PER PROCESS -----		
	Memory	Disk
Baseline code, static data and system overhead	194.0 MB	0.0 MB
BLAS internal memory storage	0.0 MB	0.0 MB
Model and support data	227.0 MB	0.0 MB
Electronic energy minimisation requirements	105.5 MB	0.0 MB
Force calculation requirements	5.6 MB	0.0 MB
	-----	-----
Approx. total storage required per process	526.5 MB	0.0 MB
Requirements will fluctuate during execution & may exceed these estimates		
+-----		

- Reported just before the main calculation starts

- Operating system's report at the end
CASTEP reports what it actually used per MPI process
(according to the operating system) at the end of a calculation

Calculation time	=	26253.18 s
Finalisation time	=	0.89 s
Total time	=	26260.79 s
Peak Memory Use	=	12308976 kB

So far we've talked about:

- Each core having its own separate parts of the data
- Will be stored in whatever RAM the core has
- Cores need to communicate to share data
- Distributed memory parallelism

CASTEP in Parallel

Introduction

Reciprocal-Space Sampling

CASTEP in Parallel

K-Points G-Vectors

Memory

Shared memory

Summary

- Common to have multi-core processors
- Each core on the same node can access shared memory (RAM)
- Each MPI processes can set up extra compute "threads" on other cores.
- Shared memory parallelism (SMP) via OpenMP
- Activated via the **Environment Variables**:
CASTEP_NUM_THREADS=<integer>
or,
OMP_NUM_THREADS=<integer>

CASTEP must have been compiled with support for threading.
On a work station/VM with e.g. 16 cores

```
export CASTEP_NUM_THREADS=4  
mpirun -np 4 castep.mpi <seedname>
```

Using a submission script on HPC systems, with e.e. 128 cores per node

```
#SBATCH -nodes=1  
#SBATCH -ntasks-per-node=32  
#SBATCH -cpus-per-task=4  
export CASTEP_NUM_THREADS=${SLURM_CPUS_PER_TASK}  
mpirun -np 32 castep.mpi <seedname>
```

OpenMP thread performance depends on many things

- Size of your problem
- Exact combination of OpenMP environment variables
- Phase of the moon...

E.g. each node of ARCHER2 has two 64-core CPUs, so 128 cores per node. What should `CASTEP_NUM_THREADS` be?

- 1, to begin with, MPI is just better.
- If your job needs too much memory, under-populate nodes and use `CASTEP_NUM_THREADS` to claw back performance.
- Performance gains will drop off above 4-8 threads, 16 max

- **k**-point parallelism is very efficient, but you eventually run out of **k**-points
- **G**-vector parallelism is good but becomes worse as you use more and more cores.
- Using threading, via `CASTEP_NUM_THREADS` can improve scaling
- Combining these allows CASTEP to scale well to many cores, but careful choice of the no. cores can improve performance considerably:
 - Know how many **k**-points you're using (`-dryrun`)
 - For phonons, use the `phonon_kpoints` tool
 - Check the parallel efficiency report at the end

NB there also band-parallelism, and some tasks allow extra 'task-farming' parallelism, which is very efficient.